Pineda Chavez Giovanni num. de cuenta: 323014302 |
Bitácora:

1. Las cuatro relaciones:
- Herencia: ("public class Caballero extends Personaje") y ("super(nombre, 120);"). El super(...) llama al constructor de la clase padre para inicializar atributos comunes antes de aplicar lo de la clase hija.
- Composición: ("this.inventario = new Inventario();"). El dueño (Personaje) hace el new; si el personaje desaparece, su inventario muere con él.
- Agregación: ("herramientas[cuantas] = h;"). Las herramientas se crean fuera y solo llegan como parámetro. Si el dueño desaparece, la herramienta sigue viva independientemente.
- Asociación: ("this.companero = companero;"). Los personajes se conocen para ayudarse, pero ninguno es parte del otro.

2. Los huecos:
- Ninguna trampa funcionó la primera vez gracias a encapsular con private y validar las reglas desde el principio, esto no hubiera sido posible sin la guía y ayuda de la IA.
- La línea ("if (estaVivo() && puntos > 0)") dentro de curar() bloquea la trampa de "golpe negativo para curarse" y "revivir a un personaje muerto".
- ¿Por qué no hay setVida ni getHerramientas? No hay setVida para evitar que un tramposo asigne vida infinita. No hay getHerramientas() devolviendo el arreglo completo porque permitiría meter una cuarta herramienta saltándose el método agregar().

3. Una prueba roja:
- Mensaje: expected: <70> but was: <50>
- Significado: 70 es lo que esperaba la prueba, 50 es lo que devolvió el código real.
- Quién estaba mal: La prueba estaba mal. El Mago empieza con 80 de vida, menos 30 de daño es 50. El código calculó perfectamente, la prueba esperaba equivocadamente un 70.

Bravo Olmedo Paulo Cesar num. de cuenta:323297291

Bitácora: En esta práctica trabajamos en un videojuego llamado "La mazmorra" con tres tipos de personajes: caballeros, magos y arqueros, los cuales usan herramientas que se desgastan.

- Cómo se organizaron las clases: Usamos herencia para que los personajes compartan acciones básicas, composición para que cada personaje cree su propio inventario, agregación para meter herramientas al inventario, y asociación para que los personajes puedan ser compañeros.
- Protección de datos: Pusimos los datos como privados (encapsulamiento) para que nadie pueda modificarlos a la fuerza. Probamos esto con archivos especiales (Tramposo.java y TrampasDeAcceso.java) y comprobamos que el programa bloquea cualquier trampa o intento de modificar los datos desde fuera.
- Pruebas JUnit: Hicimos pruebas automáticas y las 10 pasaron con éxito. También vimos que, si una prueba falla (como una que daba rojo por error en la expectativa), hay que revisar bien qué era lo que esperaba para saber si el error estaba en la prueba o en el código.
- Conclusión: Aprendí lo importante que es proteger la información de las clases y asegurarnos de que el programa cumpla siempre con sus propias reglas.

Foto salida de Main

```
pined@LAPTOP-UAOB4CKN MINGW64 ~/mpoo-2027-1/entregas/pineda_giovanni/p09 (entregas_pineda_giovanni)
$ javac -d out src/mazmorra/*.java
java -cp out mazmorra.Main
Al empezar:
Conan (Caballero) 120/120
Merlin (Mago) 80/80
Legolas (Arquero) 100/100
Despues de una ronda:
Conan (Caballero) 110/120
Merlin (Mago) 62/80
Legolas (Arquero) 75/100
Espada: durabilidad 99

pined@LAPTOP-UAOB4CKN MINGW64 ~/mpoo-2027-1/entregas/pineda_giovanni/p09 (entregas_pineda_giovanni)
$ javac -cp out -d out src/trampas/TrampasDeAcceso.java
src\trampas\TrampasDeAcceso.java:10: error: vida has private access in Personaje
        heroe.vida = 9999;
              ^
src\trampas\TrampasDeAcceso.java:11: error: inventario has private access in Personaje
        heroe.inventario = new Inventario();
              ^
src\trampas\TrampasDeAcceso.java:14: error: durabilidad has private access in Herramienta
        espada.durabilidad = 1000000;
               ^
src\trampas\TrampasDeAcceso.java:17: error: herramientas has private access in Inventario
        bolsa.herramientas[0] = espada;
              ^
src\trampas\TrampasDeAcceso.java:18: error: cuantas has private access in Inventario
        bolsa.cuantas = 50;
              ^
5 errors

pined@LAPTOP-UAOB4CKN MINGW64 ~/mpoo-2027-1/entregas/pineda_giovanni/p09 (entregas_pineda_giovanni)
$ javac -cp out -d out src/trampas/Tramposo.java
java -cp out trampas.Tramposo
BLOQUEADA  1) golpe negativo para curarse
BLOQUEADA  2) curarse arriba del tope de vida
BLOQUEADA  3) forjar una espada de 5000 de poder
BLOQUEADA  4) cargar una cuarta herramienta
BLOQUEADA  5) pedir la herramienta 99
BLOQUEADA  6) atacar sin ninguna herramienta
BLOQUEADA  7) revivir a un personaje muerto
```

```
pined@LAPTOP-UAOB4CKN MINGW64 ~/mpoo-2027-1/entregas/pineda_giovanni/p09 (entregas_pineda_giovanni)
$ javac -cp lib/junit-platform-console-standalone-1.11.4.jar -d out src/mazmorra/*.java test/mazmorra/*.java

pined@LAPTOP-UAOB4CKN MINGW64 ~/mpoo-2027-1/entregas/pineda_giovanni/p09 (entregas_pineda_giovanni)
$ java -jar lib/junit-platform-console-standalone-1.11.4.jar execute --class-path out --scan-class-path

-fÆÜ Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Platform Suite [OK]
+-- JUnit Jupiter [OK]
| '-- PersonajeTest [OK]
|   +-- laVidaNuncaPasaDeLaMaxima() [OK]
|   '-- recibirDanioRestaLosPuntos() [OK]
'-- JUnit Vintage [OK]

Test run finished after 50 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         2 tests found           ]
[         0 tests skipped         ]
[         2 tests started         ]
[         0 tests aborted         ]
[         2 tests successful      ]
[         0 tests failed          ]
```